



Spring 開発演習 説明資料

目次

1. 演習概要	2
目的	2
テーマ	2
成果物	2
2. 演習詳細	3
使用技術	3
演習形式	3
3. 演習の環境構築	5
SQL 文の実行	5
編集用プロジェクトのインポート	5
ファイル構成・実装対象ファイル	7
4. テストの実行と採点	9
採点基準	9
結合テストの実行と成果物提出	9
5. 補足	12
機能別テストの実行	12
JUnit を使用したデバッグのポイント	13
主なエラーと対処	15

1. 演習概要

目的

本演習は、研修 2 ヶ月目までに学んだ講義内容の理解度、及び実装力を測ることを目的とする。

テーマ

社員の情報を一元管理（検索、登録）する Web アプリケーションを開発することを課題とする。
実装期間として約 3 日～5 日間設ける。

成果物

演習における成果物は下記とする。

成果物
ソースコード
SQL 文
結合試験結果報告書 (surefire-report.html)
発表会資料

2. 演習詳細

使用技術

社員管理システムを構築する際に用いる技術は以下のようになる。

使用技術
Java (Oracle Java SE Development Kit 17)
Spring Boot 3.1.x
Oracle Database 21c Express Edition

アプリケーションの動作確認には「Google Chrome」を使用する。

演習形式

演習は個人で取り組み、予め提示したシステム要件に沿って実装を行う。

要件となる機能は、必須機能、追加機能の2つに分類される。

必須機能	追加機能
01 ログイン (実装済み)	06 ログイン中のユーザ名表示
02 ログアウト (実装済み)	07 入力チェック
03 社員一覧表示	
04 社員名検索	
05 社員登録	

テーブル構成は以下の 2 テーブルとする。

employee テーブル

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員 ID	emp_id	NUMBER	5 桁	PRIMARY KEY	
2	パスワード	emp_pass	VARCHAR2	16 文字	NOT NULL	
3	社員名	emp_name	VARCHAR2	30 文字	NOT NULL	
4	性別	gender	NUMBER	1 桁	NOT NULL	1:男 2:女
5	住所	address	VARCHAR2	60 文字	NOT NULL	
6	生年月日	birthday	DATE	-	NOT NULL	
7	権限	authority	NUMBER	1 桁	NOT NULL	1:一般 2:管理者
8	部署 ID	dept_id	NUMBER	2 桁	NOT NULL FOREIGN KEY	

department テーブル

No	論理名称	物理名称	データ型	桁数	制約	備考
1	部署 ID	dept_id	NUMBER	2 桁	PRIMARY KEY	
2	部署名	dept_name	VARCHAR2	15 文字	NOT NULL	1:営業部 2:経理部 3:総務部

3. 演習の環境構築

SQL 文の実行

LMS で配布する「Spring_開発演習_SQL 文」を SQLDeveloper または sqlplus で実行し、ユーザ・テーブル・レコードを作成、挿入する。

編集用プロジェクトのインポート

「Spring_開発演習(社員管理システム)_ソースコード(編集用)」を展開し、Eclipse にインポートする。

【インポート手順】

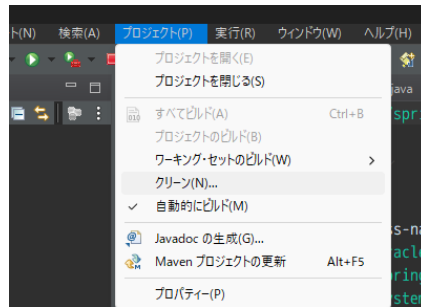
- ・ 「Spring_開発演習(社員管理システム)_ソースコード(編集用)_X.X.X.zip」を解凍して「spring_crud_name」があることを確認する
- ・ 上部タブの ファイル > インポート を選択
- ・ 一般 > 既存プロジェクトをワークスペースへ を選択
- ・ ラジオボタン：ルートディレクトリーの選択 を選択し、右の「参照」を押下
- ・ 「spring_crud_name」を選択
- ・ プロジェクトのボックス内に「spring_crud_name」が表示されるので選択し、オプションの「プロジェクトをワークスペースにコピー」にチェックが入っていることを確認する
- ・ 「完了」を押下
- ・ [spring_crud_name]を右クリックしてリファクタリングを選択し、プロジェクト名を自身の名前に変更する
例 spring_crud_sato_taro

インポート後はプロジェクトを起動し[localhost:7779/spring_crud/]にブラウザでアクセスしログイン画面が表示されることを確認する。

※Q&A プロジェクトがビルドしない場合

SpringBoot アプリケーションを起動しても Tomcat が 7779 ポートで立ち上がらない

プロジェクトをクリーンします。プロジェクト>クリーンを選択し、「すべてのプロジェクトをクリーン」または対象のプロジェクトにチェックが入っている状態でクリーンをクリックします。右下のステータスバーにクリーンの進捗が表示されます。

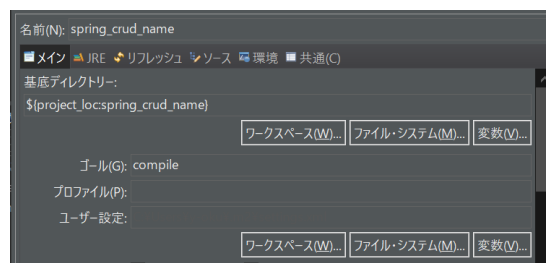


進捗の表示がなくなったら、再び SpringBoot アプリケーションを起動してください。

プロジェクトを実行すると ClassNotFoundException が発生する

Java ファイルのコンパイルがされていない。プロジェクトを右クリック→「実行→9 Maven ビルド」を選択しゴールに以下のコマンドライン引数を貼り付けて実行する。

compile



※プロジェクトを実行すると「XXX has been compiled by a more recent version of the Java Runtime～」エラー（UnsupportedClassVersionError）と表示される場合

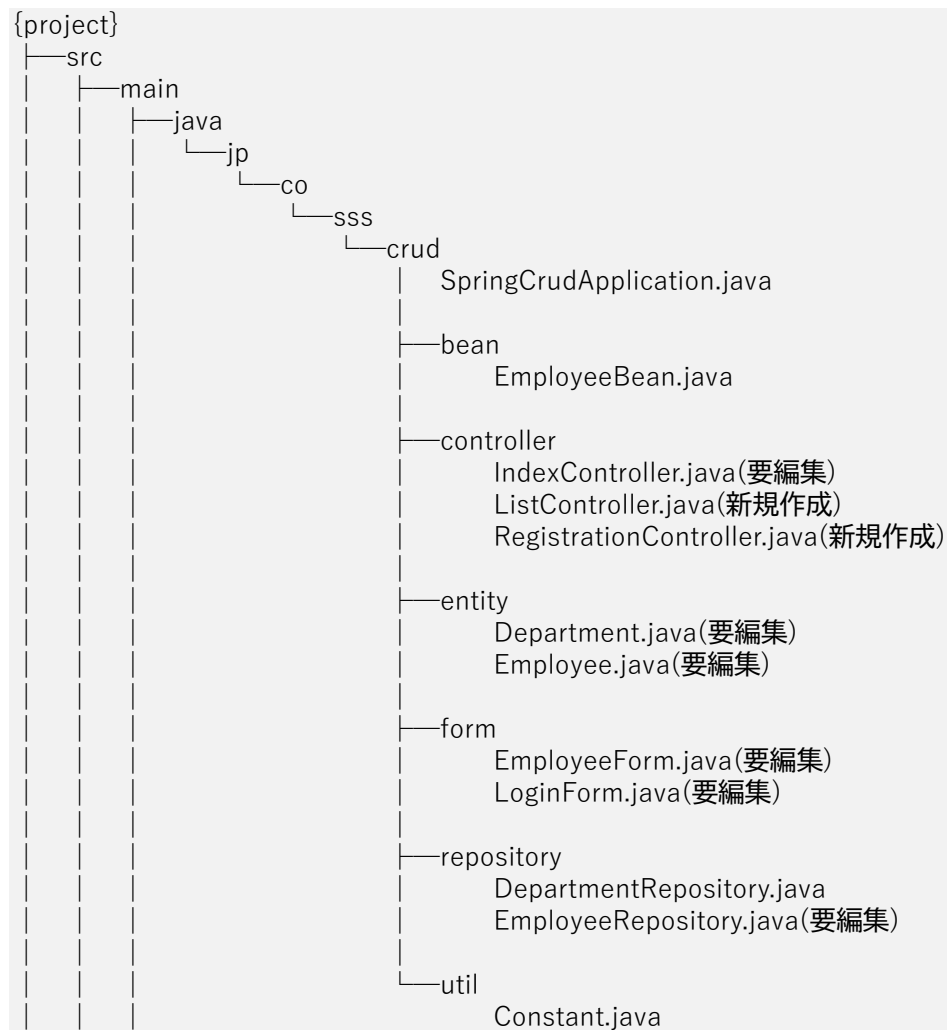
JRE のバージョンがプロジェクトと Eclipse の実行環境とで異なっている。以下の表示でバージョンが揃っているか、確認をする。

- ① プロジェクトを右クリックし「プロパティ→サイドバー：Java コンパイラー」を選択し、コンパイラー準拠レベルのバージョンを確認
 - ② pom.xml の java.version タグを確認
- ①②が演習のバージョンとずれている場合、開発演習のバージョンに合わせ修正を行う。

ファイル構成・実装対象ファイル

- ・ ファイル構成は以下とする。ファイル名のみのものは実装済みである。
- ・ Java ソースコードについては、必要に応じて新規ファイルを作成しても良い。
- ・ テスト結果に影響が出るため、**既存の HTML のタグ、属性などの構成は変えないこと**。また、新規で作成するファイルはモックや既存コードをよく参照し作成すること。
- ・ URL 設計は自身で考えること。ただし、既存のソースに記載のある URL はそのまま使用する。
 - ✧ ※例：新規社員登録 入力画面 [spring_crud/regist/input]
- ・ 要件定義書、基本設計書、詳細設計書、テスト仕様書は自身で読み込みを行い、疑問点は適宜質問して解決すること。
- ・ 開発時の環境は「application-product.properties」に記載されている。
- ・ テスト時の環境は「application-test.properties」に記載されている。なお、H2DB は組み込み DB のため、Spring Boot を再起動するたびにテーブルが初期化される。

【ファイル構成】




```
├──resources
│   ├──application.properties(テスト時編集)
│   ├──application-product.properties
│   ├──application-test.properties
│   ├──data.sql
│   ├──messages.properties
│   ├──schema.sql
│   └──ValidationMessages.properties
├──static
│   ├──css
│   │   ├──layout.css
│   │   └──stylesheet.css
│   └──js
│       └──jquery-1.10.2.min.js
├──templates
│   ├──error.html
│   ├──index.html(要編集)
│   ├──common
│   │   ├──footer.html
│   │   ├──header.html(要編集)
│   │   ├──layout_3block.html
│   │   ├──layout_4block.html
│   │   └──sidebar.html
│   ├──list
│   │   └──list.html(要編集)
│   └──regist
│       ├──regist_check.html(要編集)
│       ├──regist_complete.html(要編集)
│       └──regist_input.html(要編集)
```

4. テストの実行と採点

採点基準

各機能の完成は[src/test/java]以下にあるテストコードの結果によって採点する。

例 ログイン機能 > jp.co.sss.crud.test01.LoginTest.java

結合テストの実行と成果物提出

テストフェーズ期間に入ったら結合試験を実施する。試験は Junit および Selenium を用いての自動テストを実施する。テスト要件は「Spring_開発演習_テスト仕様書」を参照すること

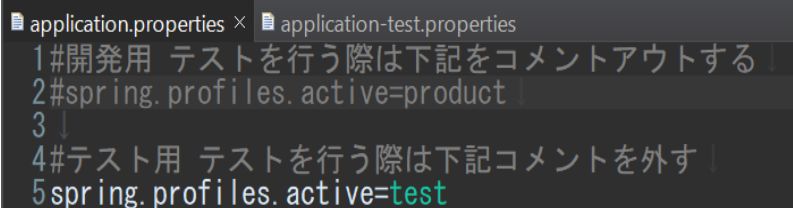
結合試験実施ならびに試験結果の提出は以下の通り行う。

1. テスト実行用のドライバファイル「chromedriver.exe」を配置する。(別紙「web アプリケーション開発演習_chrome driver 更新手順」の手順を参考にし、ドライバファイルをプロジェクトに配置する。ドライバの場所は「{project}/driver/」となる。

※注意

「chromedriver.exe」が配置されていない場合、テストの実行に失敗します。
テストを実行する前は必ず上記の「1.」の手順を確認、実施してください。

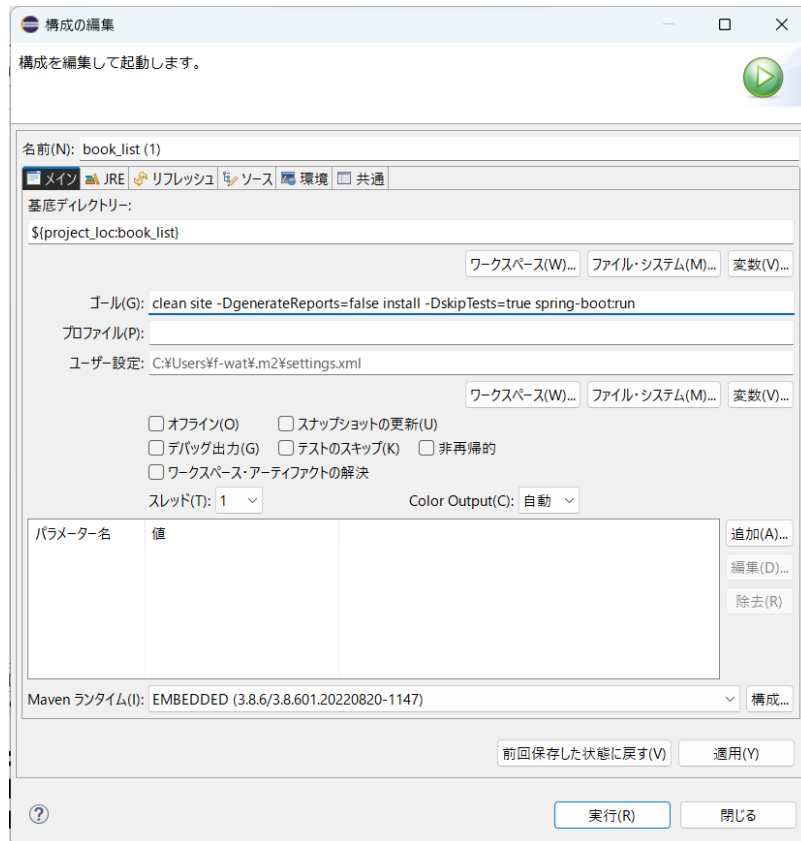
2. application.properties を修正しテスト用プロファイルに切り替える



```
application.properties × application-test.properties
1#開発用 テストを行う際は下記をコメントアウトする
2#spring.profiles.active=product
3
4#テスト用 テストを行う際は下記コメントを外す
5spring.profiles.active=test
```

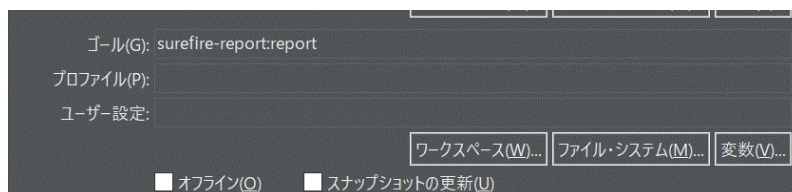
3. プロジェクトを右クリック→実行→Maven ビルド を選択しゴールに以下のコマンドライン引数を貼り付ける（レポートのテンプレートがインストールされ、プロジェクトが起動する）

clean site -DgenerateReports=false install -DskipTests=true spring-boot:run

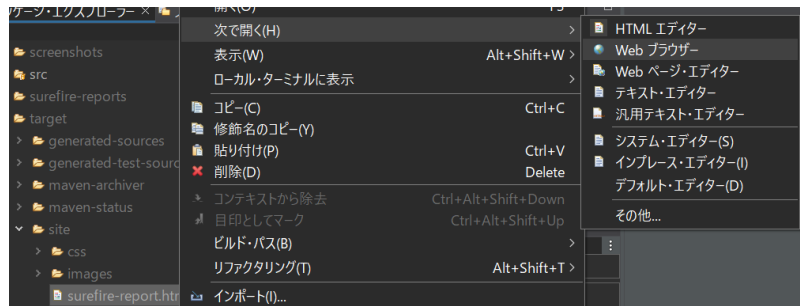


4. 同様に Maven ビルドを選択し以下のコマンドを貼り付ける。テストが実行されレポートが出力される。※テストの実行には十数分程度時間がかかる場合があります。**Eclipse** のコンソールで「**BUILD SUCCESS**」の文字が確認できたら、実行完了になります。

surefire-report:report



5. {project}/target/site/surefire-report.html をブラウザで開くとテスト結果が見られる。
ファイルを開く際は「surefire-report.html を右クリック > 次で開く > WEB ブラウザ」を選択する。



(ファイルが見えない場合は F5 キーなどでリフレッシュすること)

※再度テストする際は 1 の項目から実施しなすこと

surefire-report.html は以下のような情報が記載されている。

- ・テスト項目 (1 機能 1 パッケージ)
- ・テスト項目数
- ・実行結果 (エラー、失敗)
- ・成功率

※成功率 100%の機能を完成として採点する

Package List						
[Summary] [Package List] [Test Cases]						
Package	Tests	Errors	Failures	Skipped	Success Rate	Time
jp.co.sss.book.test04	3	0	0	0	100%	35.881
jp.co.sss.book.test05	2	0	0	0	100%	34.708
jp.co.sss.book.test02	1	0	0	0	100%	17.359
jp.co.sss.book.test03	1	0	0	0	100%	17.405
jp.co.sss.book.test00	1	0	0	0	100%	17.844
jp.co.sss.book.test01	1	0	0	0	100%	17.28
jp.co.sss.book	1	0	0	0	100%	3.675
jp.co.sss.book.test06	5	0	0	0	100%	69.883
jp.co.sss.book.test07	2	0	0	0	100%	34.707

Note: package statistics are not computed recursively, they only sum up all of its testsuites numbers.

6. {project}/target/site フォルダをコピーし、任意の場所にペーストしてから ZIP 化してサポーターに提出する。

(また、LMS にも成果物としてアップロードすること)

成果物名：社員管理システム_結合試験結果_{NAME}.zip

※{NAME}は受講生名

※担当者様への共有

LMS マニュアル # 5. ファイルアップロード機能 を参照する

※講師への共有

Slack または LMS の成果物アップロード機能を用いる

5. 補足

機能別テストの実行

機能が完成するごとにテストコードを実行し確認をする。

【進行例】

機能を実装→機能テスト NG→バグ修正→機能テスト OK→次の機能を実装

機能ごとのテストコードの実行は以下の手順で実施する。

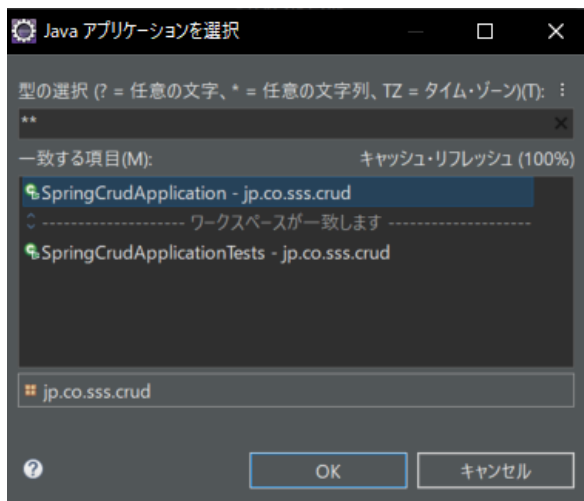
1. application.properties を修正しテスト用プロファイルに切り替える



```

application.properties × application-test.properties
1#開発用 テストを行う際は下記をコメントアウトする
2#spring.profiles.active=product
3
4#テスト用 テストを行う際は下記コメントを外す
5spring.profiles.active=test
  
```

2. Spring Boot を起動する。起動する際に選択を求められる場合は「SpringCrudApplication」を選択する。



3. 該当のテストクラスを右クリックし、「実行 > 2 JUnit テスト」を選択する

実行結果がすべて OK であれば機能完成とする。

実行結果がエラーや失敗であればソースコードに間違いがあるため、確認する。

<NG の場合>



<OK の場合>



※修正後は Spring Boot を一度停止し、1 の手順から再度テストを行う。

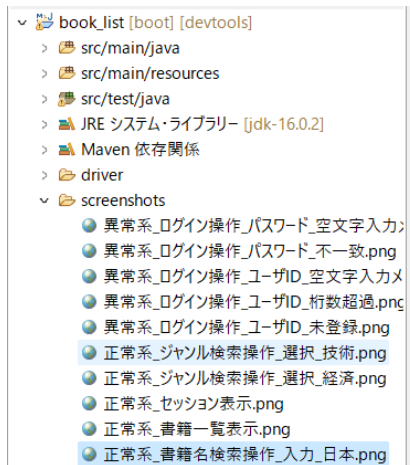
※テスト OK の後は application.properties を修正し、開発用プロファイルに戻す

JUnit を使用したデバッグのポイント

テストを行い要求される品質を満たすことは非常に重要である。テスト後のエビデンスの確認、障害トレースの例を以下に示す。参考にしてデバッグを行うこと。

エビデンスの確認

{project}/screenshots/ フォルダの中にテスト実行時のエビデンス画像が格納される

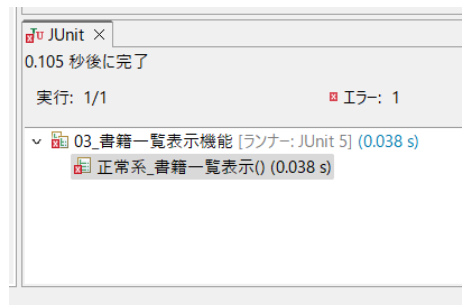


エビデンス画像が想定される画面となっているか確認する

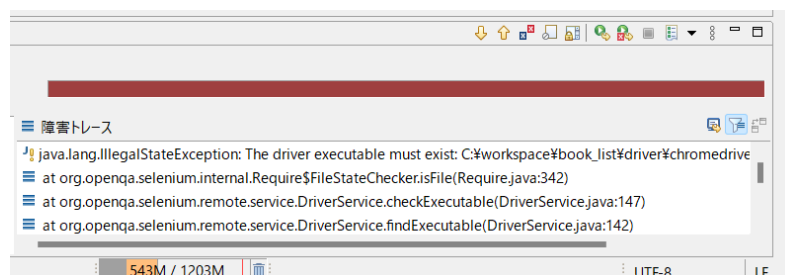
エビデンス画像がない場合は『5-2-2. Junit の障害トレース』を参照する。

Junit の障害トレース

Junit ビューにある「テストクラス > テストメソッド」を1クリックすると右部に「障害トレース」が表示され、どんなエラーが発生したのか？確認することができる。



↓ 1 クリック後



また、テストメソッドをダブルクリックするとエラーが発生したテストの箇所を開くことができる。

(下の画像では BookListTest.java の 54 行目にエラーが発生したことがわかる)

```

49 // 指定したURLに移移する
50 webDriver.get("http://localhost:2222/book_list/");
51 // 最大5秒間待つ
52 webDriver.manage().timeouts().pageLoadTimeout(5, TimeUnit.SECONDS);
53 // 表示されている要素の取得
54 WebElement loginIdElement = webDriver.findElement(By.name("bookUserId"));
55 // 取得したフォームの入力値を削除
56 loginIdElement.clear();
57 //取得したフォームにlを入力
58 loginIdElement.sendKeys("l");

```

主なエラーと対処

【原則】デバッグする際にテストクラスのコードを修正してはならない。やむを得ず修正する際は必ず講師の許可を得ること。

① 実行結果がエラー かつ NoSuchElementException が発生している

原因：HTML タグや class 属性などが想定通りになっていない可能性がある。既存コードのコメントをよく参照し修正すること。

②実行結果が失敗 かつ AssertionError が発生している

原因：ブラウザに表示される項目（Element）が想定と違う場合に発生する。障害トレースには [expected:<A> but was:] という記載がされる。A が期待される値、B が実際の値である。設計書を参照し正しい表示に修正する。以下の点も併せて確認すること。

- ・ スコープに登録されている値が想定通りかどうか？
- ・ HTML に表示される表記が正しいかどうか？
- ・ HTML に表示されるフォーマット（形式）が正しいかどうか？
- ・ 表示される項目の順序が想定通りかどうか？

